COMP3419 Assignment 2

Justin Hoogwaerts

The University of Sydney

1 Introduction

The aim of this project is to produce a model which can accurately classify images from the GTSRB (German Traffic Sign Recognition Benchmark) using Python and TensorFlow. Within this project a goal was set for the model to have an accuracy $\geq 95\%$ on set of 12629 test images.

1.1 Method

The language of choice for developing this model was Python along with the popular TensorFlow and Keras libraries, which provided much of the back-end for compiling and setting up the model layer by layer. Data manipulation was performed using Pandas. For evaluating the model the given evaluation.py, test.py and testloader.py had to be completely changed as they supported PyTorch models and were not suitable for evaluating the effectiveness of .keras models

1.2 Dataset

The GTSRB is a dataset which contains 51839 images, 39209 being for training and validation and 12630 being for testing. The University has provided a csv file conatining the **ClassId** for the test image set which are not given with the standard GTSRB data set.

1.3 Findings

The findings were that it was surprisingly easy to train a model which accurately performed a multi-class image classification on the GTSRB. The model demonstrated in this report achieved an accuracy of 98% which is comparable to other popular models at performing the same task.

2 Methods

2.1 Data Extraction

For any model to be trained the data first needed to be loaded from the file structure within the GTSRB data set. As the $GTSRB/Final_Training$ data contained 43 folders named 000XX where XX represented the class number of the images, all images under the same folder represented the same type of road sign. E.G (Crossing). Thus iterating over all the images under a folder and adding them to an array whilst also adding the name of the folder to another array named labels returned the data.

2.2 Image Preprocessing

To clean the data all the images were converted to (32×32). Converting the images to (32×32) was done to standardize the inputs to the model as well as to speed up the processing time. The images were also flipped from BGR to RGB to train the model on data which more closely resembles the real world, this may prove useful if the model was ever used in a real-time application where the process of converting the camera input from RGB to BGR before feeding it to the model would be eliminated saving time.

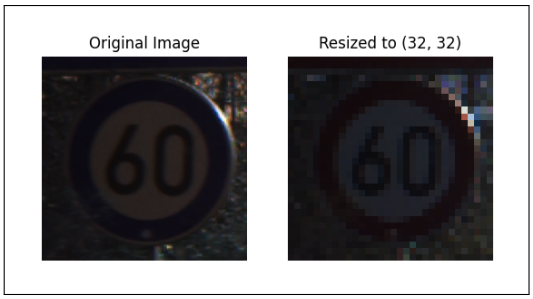
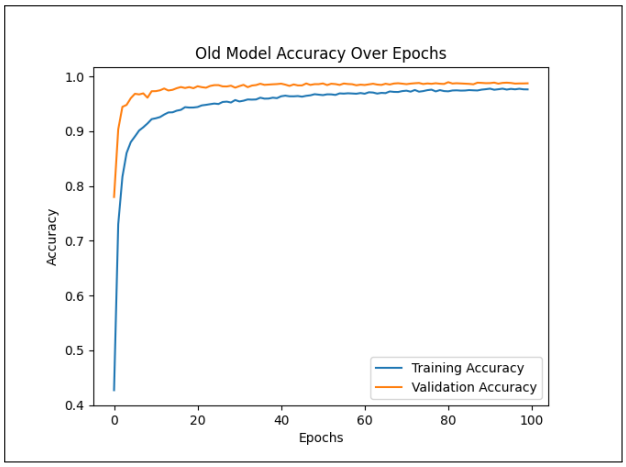


Figure 1: *Image before and after processing*

2.3 Neural Networks

The neural network in this assignment went through two iterations the first one only averaged an accuracy of roughly 93% with the following accuracy during training epochs. The model contained 7 layers starting with a *Conv2D* layer and ending with a *Dense* layer with 43 nodes matching the class count of the GTSRB dataset.



The experimental model also had significantly lower class accuracy in the classes which both models struggled with.

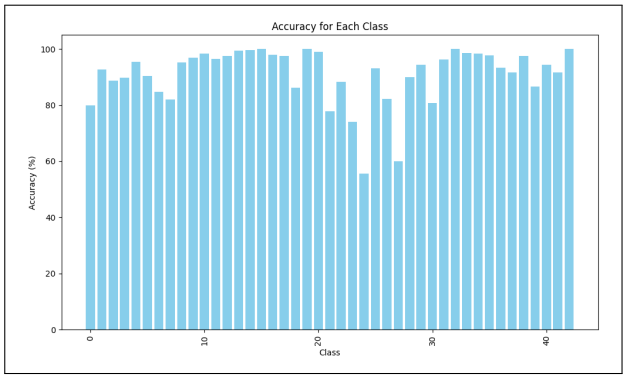


Figure 2: *Class by Class Accuracy for Experimental Model*

The results of the updated model are in the results section of this report however it significantly improves on the weaknesses of the experimental model reaching an average accuracy of 98%. The newer model takes roughly 4 minutes to train compared to the 1.7 minutes of the experimental model however both models use 100 epochs to train.

3 Experimental Setup

3.1 Dataset

The dataset is the GTSRB (German Traffic Sign Recognition Benchmark). Which contains 51839 images of road signs. Originally the dataset does not provide **ClassId** labels for the 12630 test images which would limit the images that could be used for training as the 39209 would have to be split into training, validation and testing groups. However since the university provided the **ClassId's** for 12629 test images (missing the final one for some reason) the 39209 training images were only split into training and validation groups with a $80/20$ split respectively.

3.2 Evaluation Metrics

The evaluation metrics used to quantify the performance of the model were the 12629 test images and corresponding labels. These were loaded into an image array and underwent the same preprocessing as the training images for consistency. The evaluation code given within the scaffold was changed to accommodate a TensorFlow/Keras model, the *model.py* code also calculates the test accuracy after training the model the same way that the modified *evaluation.py* does

4 Results & Discussion

4.1 Experimental Model

The experimental model, which featured 7 layers in the form

Conv2D (16 filters, 3x3 kernel) -> MaxPooling2D (2x2 pool size) -> Dropout (0.2) -> Flatten -> Dense (64 units, ReLU) -> Dropout (0.3) -> Dense (43 units, softmax)

Had an average $\approx 93\%$ accuracy of classifying images in the testing set. Both models struggled with classes 24 and 27 as seen in **Figure 2** with the accuracy $\approx 60\%$ for this model in both classes. Examining the images within class 24 we see images like this.



Figure 3: *00000_00004.ppm*

Which we can compare to images from classes which the model did perform well in like class 19. This allows us to begin to understand the factors regarding image quality which affect the models performance.



Figure 4: *00003_00011.ppm*

We can notice that the lack of accuracy may be due to poor color contrast in **Figure 3**. The low accuracy could also be attributed to the image resolution as the image shown in **Figure 3** has a resolution of (28×27) lower than the input resolution of the model at (32×32).

4.2 New Model

The new and **current** model features 14 layers in the form:

1. Conv2D (64 filters, 3x3 kernel, ReLU)
2. BatchNormalization
3. MaxPooling2D (2x2 pool size)
4. Conv2D (128 filters, 3x3 kernel, ReLU)
5. BatchNormalization
6. MaxPooling2D (2x2 pool size)
7. Conv2D (256 filters, 3x3 kernel, ReLU)
8. BatchNormalization
9. MaxPooling2D (2x2 pool size)
10. Dropout (0.5)
11. Flatten
12. Dense (512 units, ReLU)
13. Dropout (0.5)
14. Dense (43 units, softmax)

This model had an accuracy on the test data of $\approx 98\%$. Each epoch took $\approx 3s$ to train with there being 100 epochs. The model's accuracy improved rapidly over the first 5 epochs as seen in the graph below. The main reason for the improved accuracy of the new model is the deeper CNN (Convolutional Neural Network)

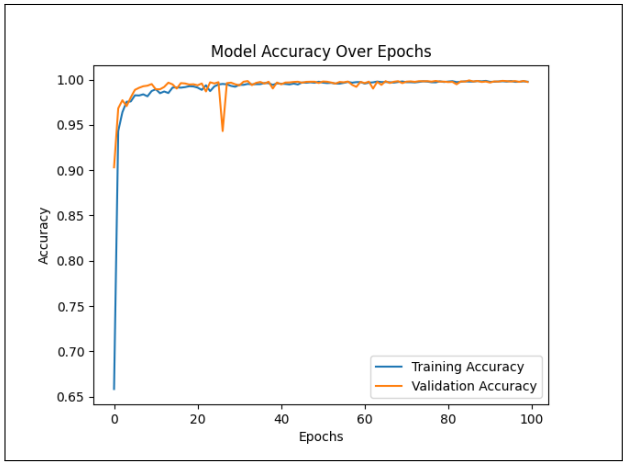


Figure 5: *Accuracy per Epoch*

As seen in the graph the **validation accuracy** starts off inconsistent but quickly stabilizes around the 40th epoch. The main benefit of the new model over the experimental model only becomes evident when examining its class by class accuracy.

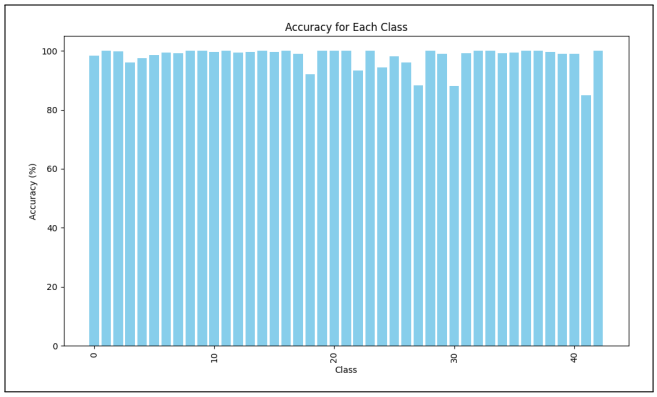


Figure 6: *Class by Class Accuracy for Improved Model*

As seen in the above graph the weak classes 24 and 27 have dramatically improved in accuracy from $\approx 60\% \rightarrow \approx 90\%$. This increases the robustness of the model rather than focusing on improving classes the model was already effective at classifying, this could possibly prove beneficial in real-time scenarios for autonomous vehicles.

5 Conclusion

The main findings from this study are the affect of contrast and image resolution on CNN model training, through examining the difference in accuracy per epoch between the experimental and new model it is evident that improving accuracy in low accuracy classes is far more important for model robustness than improving classes the model can already classify. These low accuracy classes within this model can be attributed to the low color contrast and low image res-

olution, which can be greatly improved through the use of more layers and the tweaking of dropout rates.

Bibliography

Addison, Automatic (2021). *How to Detect and Classify Road Signs Using TensorFlow*. Accessed: 2024-11-2. URL: https://automaticaddison.com/how-to-detect-and-classify-road-signs-using-tensorflow/.

May, Lawrence (2021). *Using TensorFlow for Classification on the GTSRB (German Traffic Sign Dataset)*. Accessed: 2024-11-4. URL: https://medium.com/@lawrence.may/using-tensorflow-for-classification-on-the-gtsrb-german-traffic-sign-dataset-d71e5557a015.